

Tutorial - Semana 3, Encontro 2

Introdução

No Encontro 1, o nível de teste ganhou terreno esculpido com o addon Terrain3D e um material aplicado ao `Chao`, resolvendo o problema de aparência de superfície e relevo. Faltava resolver como a luz se propaga entre essas superfícies de forma crível e em tempo real — problema que toda engine moderna precisa endereçar, sob pena de produzir cenas com sombras e reflexos artificiais — e como transformar esse projeto editável em um executável distribuível fora do editor. Este encontro resolve as duas etapas finais do Módulo 1: **iluminação global dinâmica** (SDFGI/VoxelGI) e **exportação de projeto**.

Este tutorial dá continuidade direta ao Encontro 1 — o terreno e o material já devem existir na Scene, com o Player posicionado sobre a área esculpida, antes de começar.

Objetivos da semana

- Explicar iluminação global dinâmica (SDFGI/VoxelGI) como conceito universal de renderização moderna.
- Discutir a ausência de geometria virtualizada no Godot como ponto de comparação arquitetural.
- Explicar o pipeline de exportação e gerar o primeiro build executável do Vertical Slice.

Resultado esperado ao final da semana

Ao final da Semana 3 (Encontros 1 e 2), cada estudante terá um nível de teste com terreno, material e iluminação global ativa, exportado como o primeiro build executável do Vertical Slice — encerrando o Módulo 1 com o Checkpoint de primeiro build jogável. Este tutorial cobre apenas o **Encontro 2**: a ativação da iluminação global e a exportação do projeto.

Pré-requisitos

- Terreno esculpido com Terrain3D e material aplicado ao `Chao`, com o Player posicionado sobre a área plana do terreno (ver Tutorial - Semana 3, Encontro 1).
 - Export Templates da versão do Godot em uso instalados na máquina (ver **Editor > Manage Export Templates**).
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Encontro 1, com terreno, material e Player posicionado corretamente sobre a área esculpida.

Arquivos necessários

- Nenhum arquivo externo adicional. A iluminação global é ativada nas propriedades do WorldEnvironment/Terrain3D já existentes, e a exportação depende apenas dos Export Templates instalados.

Assets utilizados

- Nenhum asset novo. A iluminação e a exportação atuam sobre o nível já modelado no Encontro 1.

Projeto esperado

- Projeto aberto no Godot 4.7, com o nível de teste completo do Encontro 1.
- Export Templates instalados e verificados em **Editor > Manage Export Templates** antes do início do laboratório.

Imagem sugerida

Objetivo: mostrar a janela **Project > Export**, com um preset de exportação já configurado para a plataforma de destino. Enquadramento: captura de tela da janela Export do Godot, lista de presets à esquerda e opções de configuração à direita. Elementos importantes: botão "Add..." para criar um novo preset, botão "Export Project" no rodapé. Destaque: o botão "Export Project", usado para gerar o build final. Legenda sugerida: "Janela de exportação com o preset configurado para o primeiro build do Vertical Slice."

Parte 1 — Iluminação global dinâmica (SDFGI/VoxelGI)

Objetivo

Entender como uma engine resolve a propagação de luz entre superfícies em tempo real, antes de ativar qualquer configuração no editor.

Conceito

Um nível com apenas uma luz direta parece plano e artificial: superfícies fora do alcance direto da luz ficam completamente escuras, mesmo quando deveriam receber luz refletida por paredes e pelo chão próximos. Resolver esse problema exige simular, de alguma forma, como a luz "salta" entre superfícies — a chamada **iluminação global**. Calcular isso com precisão total, a cada frame, é computacionalmente inviável em tempo real; por isso, engines modernas oferecem aproximações otimizadas desse cálculo.

O Godot oferece duas soluções alternativas para esse mesmo problema: **SDFGI** (Signed Distance Field Global Illumination), que aproxima a geometria da cena por campos de distância com sinal para calcular a propagação de luz sem exigir dados pré-calculados, e **VoxelGI**, que discretiza a cena em voxels dentro de um volume delimitado, com maior precisão em áreas menores e mais controladas. Ambas resolvem o mesmo problema — luz indireta crível em tempo real — com trade-offs diferentes de escala e precisão.

Passo a passo

1. Reabra o projeto do Vertical Slice e a Scene `level_exploration.tscn`, confirmando que o terreno e o material do Encontro 1 estão presentes.
2. No painel Scene, verifique se já existe um Node `WorldEnvironment` na hierarquia; caso não exista, clique com o botão direito sobre `NívelTeste` e adicione um Node do tipo **WorldEnvironment**.
3. Com o `WorldEnvironment` selecionado, no Inspector, crie um novo recurso de **Environment** (caso ainda não exista) e salve-o em `resources/` (por exemplo, `resources/environment_exploration.tres`).
4. Dentro do Environment, localize a seção **SDFGI** e habilite-a marcando a caixa **Enabled**.
5. Ajuste o parâmetro de escala/tamanho de célula do SDFGI para um valor compatível com a extensão do terreno esculpido no Encontro 1 (valores menores aumentam a precisão em áreas pequenas).

6. Rode a Scene com F6 e observe visualmente como as superfícies próximas ao `Chao` e ao terreno passam a receber luz indireta, mesmo fora do alcance direto da luz principal.
7. Compare o resultado com o SDFGI temporariamente desabilitado (desmarque e marque novamente a caixa **Enabled**), observando a diferença de iluminação nas áreas de sombra.
8. Ajuste, se necessário, a intensidade da luz principal (`LuzPrincipal` , existente desde a Semana 1) para equilibrar a cena — SDFGI amplifica a luz existente, não a substitui.
9. Salve a Scene (**Ctrl+S**).

Resultado esperado

O nível de teste possui um Node `WorldEnvironment` com um recurso `Environment` salvo em `resources/` , com **SDFGI habilitado**, produzindo luz indireta visível nas áreas de sombra do terreno e do `Chao` .

Verificando

1. Confirme, no Inspector do `Environment` , que a caixa **SDFGI > Enabled** está marcada.
2. Rode a Scene com F6 e observe se áreas fora do alcance direto da `LuzPrincipal` ainda recebem alguma luz indireta, em vez de ficarem completamente escuras.
3. Compare visualmente o nível com o SDFGI ativado e desativado, confirmando que a diferença é perceptível.

Problemas comuns

- SDFGI habilitado, mas sem efeito visível: confirmar que o parâmetro de escala/célula está compatível com o tamanho do terreno — um valor muito grande ou muito pequeno em relação ao nível pode tornar o efeito imperceptível.
- Cena com iluminação excessivamente clara ou estourada após habilitar SDFGI: reduzir a intensidade da `LuzPrincipal` , já que o SDFGI amplifica a luz indireta a partir da luz direta existente.
- Recurso `Environment` criado como embutido na Scene, em vez de salvo como `.tres` externo: sempre usar **Save As** para tornar o recurso reutilizável em outras Scenes do projeto.

Boas práticas

- Sempre comparar visualmente a cena com a iluminação global ativada e desativada antes de considerar o ajuste concluído — a diferença deve ser perceptível, não apenas teórica.
- Salvar o recurso `Environment` como arquivo externo em `resources/` , seguindo a mesma lógica de separação de dados usada para materiais e Terrain3D Storage.

- Ajustar a intensidade da luz principal em conjunto com o SDFGI, nunca isoladamente — os dois parâmetros se influenciam mutuamente.

Comparação com Unity

A Unity resolve iluminação global através do sistema de **Global Illumination** dos pipelines URP/HDRP, oferecendo tanto soluções em tempo real quanto soluções pré-calculadas via **Lightmapping** — uma abordagem híbrida distinta do SDFGI/VoxelGI do Godot, que são inteiramente dinâmicos e não dependem de bake prévio. Ambas as engines, no entanto, compartilham o mesmo objetivo conceitual: aproximar, com custo controlado, a propagação de luz indireta entre superfícies em tempo real.

Parte 2 — Geometria virtualizada: um limite compartilhado

Objetivo

Entender por que nem Godot nem Unity resolvem, nativamente, o problema de renderizar geometria extremamente detalhada sem custo proporcional de desempenho.

Conceito

Alguns motores modernos — notavelmente a Unreal Engine, com o **Nanite** — resolvem o problema de renderizar malhas com bilhões de polígonos sem exigir que a equipe de arte produza manualmente múltiplos níveis de detalhe (LODs) para cada asset. Essa tecnologia é chamada de **geometria virtualizada**: a engine decide, em tempo real, qual nível de detalhe de cada malha deve ser efetivamente desenhado, de forma transparente à equipe.

Nem o Godot nem a Unity possuem uma solução nativa equivalente. Ambas as engines dependem de técnicas tradicionais — LOD manual, impostors e occlusion culling — para controlar o custo de geometria complexa. Essa é uma discussão puramente conceitual, sem implementação prática nesta disciplina: o objetivo é que a turma reconheça esse como um limite real e compartilhado entre as duas engines estudadas, não uma desvantagem exclusiva do Godot.

Passo a passo

Esta parte não possui etapas de implementação no editor — é uma discussão conceitual conduzida em aula.

1. Revisar com a turma o que o Nanite resolve na Unreal Engine (renderização de geometria extremamente detalhada sem LOD manual).
2. Confirmar que nem o Godot nem a Unity oferecem uma solução nativa equivalente.
3. Relacionar as técnicas tradicionais disponíveis em ambas as engines (LOD manual, impostors, occlusion culling) como as alternativas atualmente usadas para esse mesmo problema.
4. Registrar essa limitação como um ponto de comparação arquitetural válido para a apresentação técnica final da Semana 17 (ver PROJECT_ARCHITECTURE.md, seção 12).

Resultado esperado

A turma reconhece a ausência de geometria virtualizada como um limite técnico compartilhado entre Godot e Unity, sem tentar implementar nenhuma solução alternativa dentro do escopo desta disciplina.

Verificando

1. Confirmar, em discussão, que os estudantes conseguem explicar o que o Nanite resolve, sem entrar em detalhes de implementação da Unreal.
2. Confirmar que os estudantes citam corretamente LOD manual, impostors e/ou occlusion culling como as alternativas disponíveis em Godot e Unity.

Problemas comuns

- Tratar a ausência de geometria virtualizada como uma falha exclusiva do Godot: reforçar que a Unity compartilha exatamente a mesma limitação.
- Tentar implementar LOD manual ou occlusion culling avançado neste encontro: essa discussão é conceitual — implementação de otimização de geometria está fora do escopo desta semana e não faz parte do roadmap do Módulo 1.

Boas práticas

- Manter essa discussão breve e conceitual, sem consumir tempo de laboratório destinado à iluminação global e à exportação.
- Conectar essa comparação diretamente à tabela de comparações arquiteturais do PROJECT_ARCHITECTURE.md (seção 12), reforçando que o documento deve ser expandido,

não contradito.

Comparação com Unity

Como discutido acima, a Unity não possui um equivalente direto ao Nanite da Unreal Engine, assim como o Godot. Ambas as engines dependem de LOD manual, impostors e occlusion culling para controlar o custo de geometria complexa — uma limitação compartilhada, e não um ponto de desvantagem exclusivo de nenhuma das duas.

Parte 3 — Pipeline de exportação e o primeiro build executável

Objetivo

Entender o que diferencia um projeto editável de um executável distribuível, antes de configurar qualquer preset de exportação.

Conceito

Um projeto Godot, enquanto aberto no editor, depende do próprio editor para rodar. Para que qualquer pessoa fora do time de desenvolvimento jogue o projeto, ele precisa ser **empacotado** em um executável autocontido — processo chamado de **exportação**. Esse empacotamento depende de **Export Templates**: versões pré-compiladas do motor Godot, específicas para cada plataforma de destino (Windows, Linux, macOS, entre outras), que substituem o editor pelo runtime mínimo necessário para rodar o jogo.

Sem os Export Templates corretos instalados, a exportação falha antes mesmo de começar — por isso a verificação da instalação é o primeiro passo prático desta etapa, não um detalhe menor.

Passo a passo

1. Antes de qualquer configuração, abra **Editor > Manage Export Templates** e confirme que os templates da versão do Godot em uso estão instalados; caso não estejam, realize a instalação antes de prosseguir.
2. Abra **Project > Export** para acessar a janela de configuração de exportação.

3. Clique em **Add...** e selecione a plataforma de destino usada pelo laboratório (por exemplo, Windows Desktop ou Linux/X11).
4. No painel de configuração do preset recém-criado, mantenha as opções padrão para este primeiro build, ajustando apenas o nome do preset para algo identificável (por exemplo, "Build Semana 3").
5. Confirme, na aba **Resources**, que todos os recursos usados pela Scene (terreno, materiais, Environment) estão marcados para exportação — a configuração padrão geralmente já cobre isso, mas vale conferir.
6. Clique em **Export Project**, escolha uma pasta de destino (por exemplo, `builds/semana_03/`) e nomeie o executável de forma identificável.
7. Aguarde a conclusão da exportação, acompanhando o progresso na própria janela.
8. Navegue até a pasta de destino e execute o arquivo gerado diretamente, **fora do editor Godot**.
9. Confirme que o nível de teste carrega corretamente, com terreno, material e iluminação global visíveis, e que o Player responde normalmente ao Input Map configurado na Semana 2.
10. Caso algo falhe apenas no executável (e funcione no editor), retorne à aba **Resources** da janela de exportação e verifique se algum asset ficou de fora do pacote de exportação.

Resultado esperado

Um executável autocontido do Vertical Slice existe fora do editor Godot (por exemplo, em `builds/semana_03/`), rodando o nível de teste completo — terreno, material, iluminação global e Player controlável — sem depender do editor para funcionar.

Verificando

1. Confirme que o arquivo executável foi gerado na pasta de destino escolhida.
2. Execute o arquivo diretamente (duplo clique ou linha de comando), fora do Godot, e confirme que o nível carrega sem erros.
3. Movimente o Player no executável, confirmando que o Input Map e a colisão com o terreno funcionam da mesma forma que no editor.

Problemas comuns

- Export Templates não instalados ou desatualizados, impedindo a exportação: sempre verificar a instalação em **Editor > Manage Export Templates** antes de configurar qualquer preset.
- Build roda no editor mas falha ao ser exportado (tela preta, assets ausentes): revisar a aba **Resources** do preset de exportação, garantindo que nenhum recurso ficou de fora do pacote.
- Caminho de destino da exportação com caracteres especiais ou espaços, causando falha silenciosa em algumas plataformas: preferir caminhos simples, sem acentuação ou espaços, para a pasta de build.

Boas práticas

- Sempre testar o executável exportado fora do editor antes de considerar a etapa concluída — um projeto que só funciona no editor não é um build validado.
- Nomear presets de exportação de forma identificável (`Build Semana 3`), facilitando reexportações futuras nas semanas seguintes.
- Manter os builds exportados organizados por semana (`builds/semana_03/`), evitando sobrescrever builds anteriores sem necessidade.

Comparação com Unity

A Unity resolve o mesmo problema através do **Build Settings**, que também depende de módulos de plataforma instalados previamente (equivalentes conceituais aos Export Templates do Godot) antes de permitir a geração de um build para uma plataforma específica. O fluxo conceitual é o mesmo em ambas as engines — configurar um preset/target, garantir os módulos de plataforma corretos e gerar um executável autocontido —, ainda que os nomes e a organização das telas de configuração sejam diferentes.

Ao final da semana

Ao final da Semana 3 (Encontros 1 e 2), o projeto do Vertical Slice deve conter:

- O Player (CharacterBody3D) controlável, montado nas Semanas 1 e 2.
- Terreno esculpido com Terrain3D e material aplicado ao `Chao` (Encontro 1).
- Iluminação global ativa via SDFGI, configurada em um `WorldEnvironment` salvo como recurso externo (Encontro 2).
- Um executável exportado e testado fora do editor, contendo todo o nível de teste funcional (Encontro 2).

Segundo o PROJECT_ARCHITECTURE.md (seção 6, Módulo 1), este resultado corresponde à conclusão dos itens "Cena de teste (graybox)" e "Renderização moderna e build", encerrando o Módulo 1 com o produto esperado: "primeiro build executável, com o jogador explorando um graybox do ambiente externo". Este encontro encerra também a Unidade I — Aprender a Ferramenta.

Desafio

Não há desafio de implementação livre neste encontro — o próprio Checkpoint (build exportado e funcional) é o entregável de encerramento do módulo. Como exercício opcional, cada estudante pode

testar um segundo preset de exportação para uma plataforma diferente da usada no Checkpoint (por exemplo, Linux além de Windows), comparando o processo entre plataformas.

Checklist

- ☐ Node `WorldEnvironment` presente na Scene, com recurso `Environment` salvo em `resources/`
- ☐ SDFGI habilitado e com efeito de luz indireta visível no terreno e no `Chao`
- ☐ Discussão conceitual sobre geometria virtualizada (Nanite) e sua ausência em Godot e Unity realizada
- ☐ Export Templates verificados e instalados antes da configuração do preset
- ☐ Preset de exportação criado em **Project > Export**, com nome identificável
- ☐ Build exportado para `builds/semana_03/` (ou pasta equivalente) e testado fora do editor
- ☐ Player controlável e terreno funcionando corretamente no executável, fora do editor

Glossário

- **SDFGI (Signed Distance Field Global Illumination):** solução de iluminação global do Godot baseada em campos de distância com sinal, sem exigir bake prévio.
- **VoxelGI:** solução alternativa de iluminação global do Godot baseada em discretização por voxels, mais precisa em volumes menores e delimitados.
- **WorldEnvironment:** Node do Godot que define as configurações globais de ambiente e iluminação da Scene, incluindo SDFGI/VoxelGI.
- **Geometria virtualizada:** técnica (como o Nanite da Unreal Engine) que renderiza malhas extremamente detalhadas sem custo proporcional de desempenho; ausente tanto no Godot quanto na Unity.
- **Export Templates:** versões pré-compiladas do runtime do Godot, específicas para cada plataforma de destino, necessárias para gerar um build exportado.
- **Exportação:** processo de empacotar um projeto editável em um executável autocontido, capaz de rodar fora do editor.

Referências

- Godot Documentation — Global Illumination (SDFGI/VoxelGI):
https://docs.godotengine.org/en/stable/tutorials/3d/global_illumination/index.html

- Godot Documentation — Exporting Projects:
<https://docs.godotengine.org/en/stable/tutorials/export/index.html>
- Godot Documentation — Standard Material 3D:
https://docs.godotengine.org/en/stable/tutorials/3d/standard_material_3d.html
- Terrain3D (addon) — Repositório oficial: <https://github.com/TokisanGames/Terrain3D>
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — Global Illumination:
<https://docs.unity3d.com/Manual/realtime-gi-using-enlighten.html>
- GDQuest: <https://www.gdquest.com/>